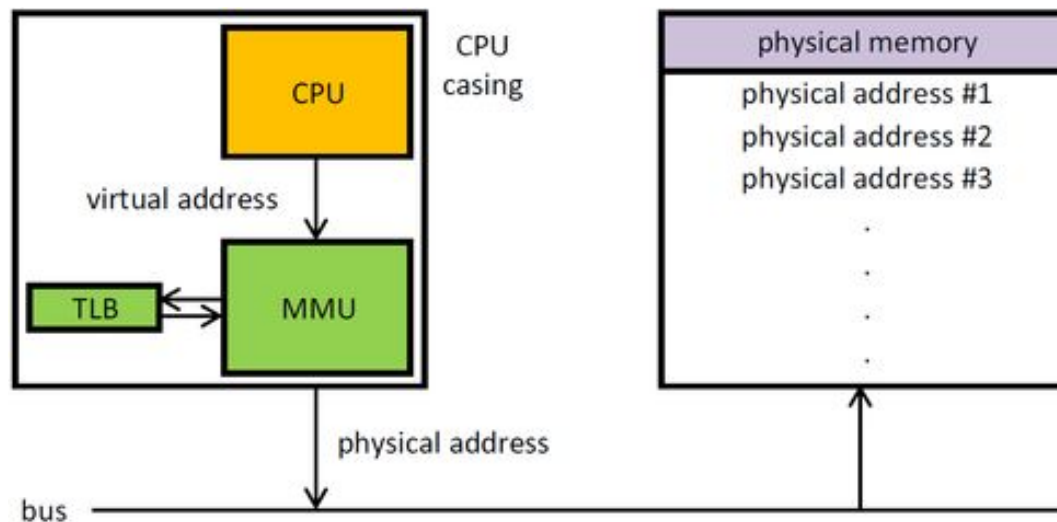


Chapter 3

Memory Management

Translation Lookaside Buffer

- Observation
 - Only a small fraction of the PTEs are heavily read; the rest are barely used at all
- A TLB is part of the chip's MMU, and is simply a hardware cache of popular virtual-to-physical address translations

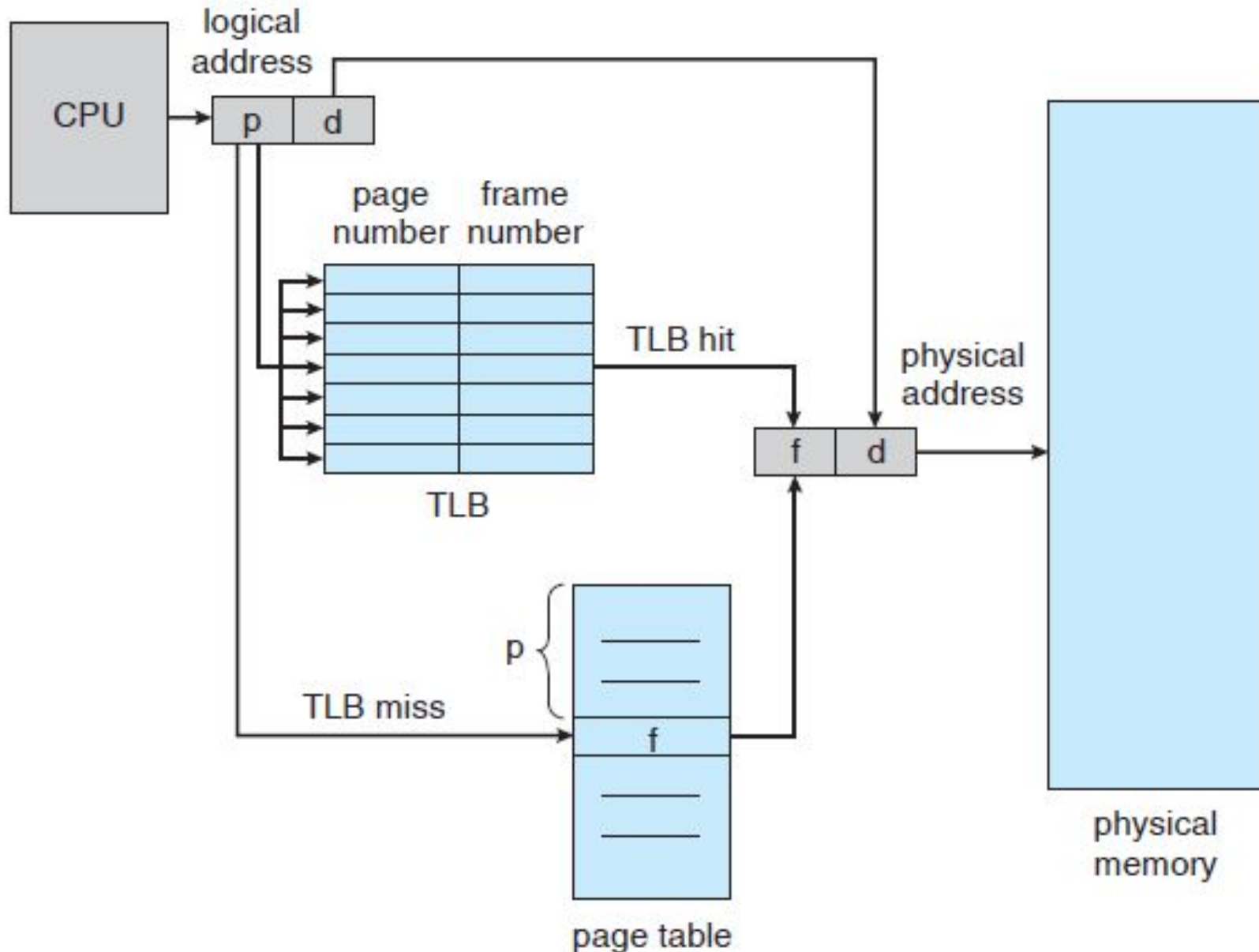


CPU: Central Processing Unit
MMU: Memory Management Unit
TLB: Translation lookaside buffer

TLB Function

- If a virtual address is presented to MMU, the hardware checks TLB by comparing all entries simultaneously (in parallel).
- If match is valid, the page is taken from TLB without going through page table.
- If match is not valid
 - MMU detects miss and does an ordinary page table lookup.
 - It then evicts one page out of TLB and replaces it with the new entry, so that next time that page is found in TLB.

Translation Lookaside Buffer



Bits in a TLB Entry

- Common (necessary) bits
 - Virtual page number: match with the virtual address
 - Physical page number: translated address
 - Valid
 - Access bits: kernel and user (nil, read, write)
- Optional (useful) bits
 - Process tag
 - Reference
 - Modify
 - Cacheable

Handling large virtual address spaces

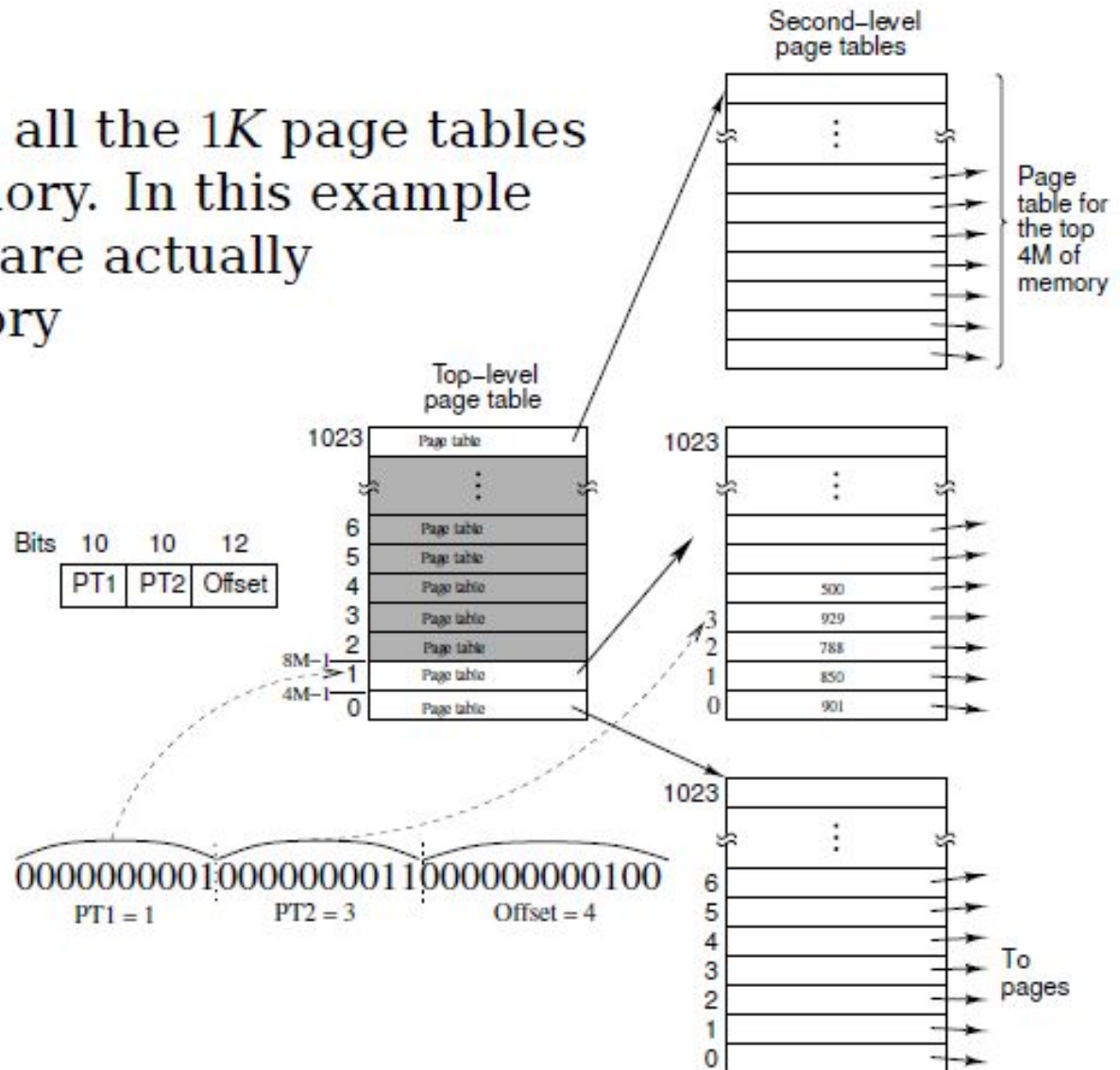
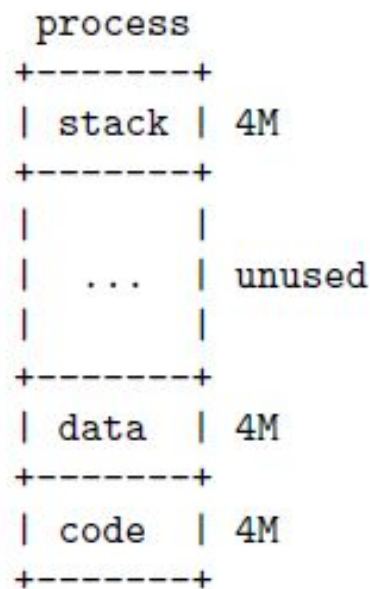
- all modern computers use virtual addresses of at least 32 bits, with 64 bits becoming increasingly common
- With, say, a 4-KB page size, a 32-bit address space has 1 million pages
- With 1 million pages in the virtual address space, the page table must have 1 million entries.
- each process needs its own page table
 - because it has its own virtual address space

Multilevel Page Tables

- Since the page table can be very large, one solution is to page the page table
- Divide the page number into
 - An index into a page table of second level page tables
 - A page within a second level page table
- Advantage
 - No need to keeping all the page tables in memory all the time
 - Only recently accessed memory mapping need to be kept in memory, the rest can be fetched on demand

Two-Level Page Tables

Don't have to keep all the 1K page tables (1M pages) in memory. In this example only 4 page tables are actually mapped into memory



Problem With 64-bit Systems

Given:

- ▶ virtual address space = 64 *bits*
- ▶ page size = 4 *KB* = 2^{12} *B*

How much space would a simple single-level page table take?

- ▶ Each page table entry takes 4 *Bytes*, then
- ▶ The whole page table (2^{64-12} entries) will take

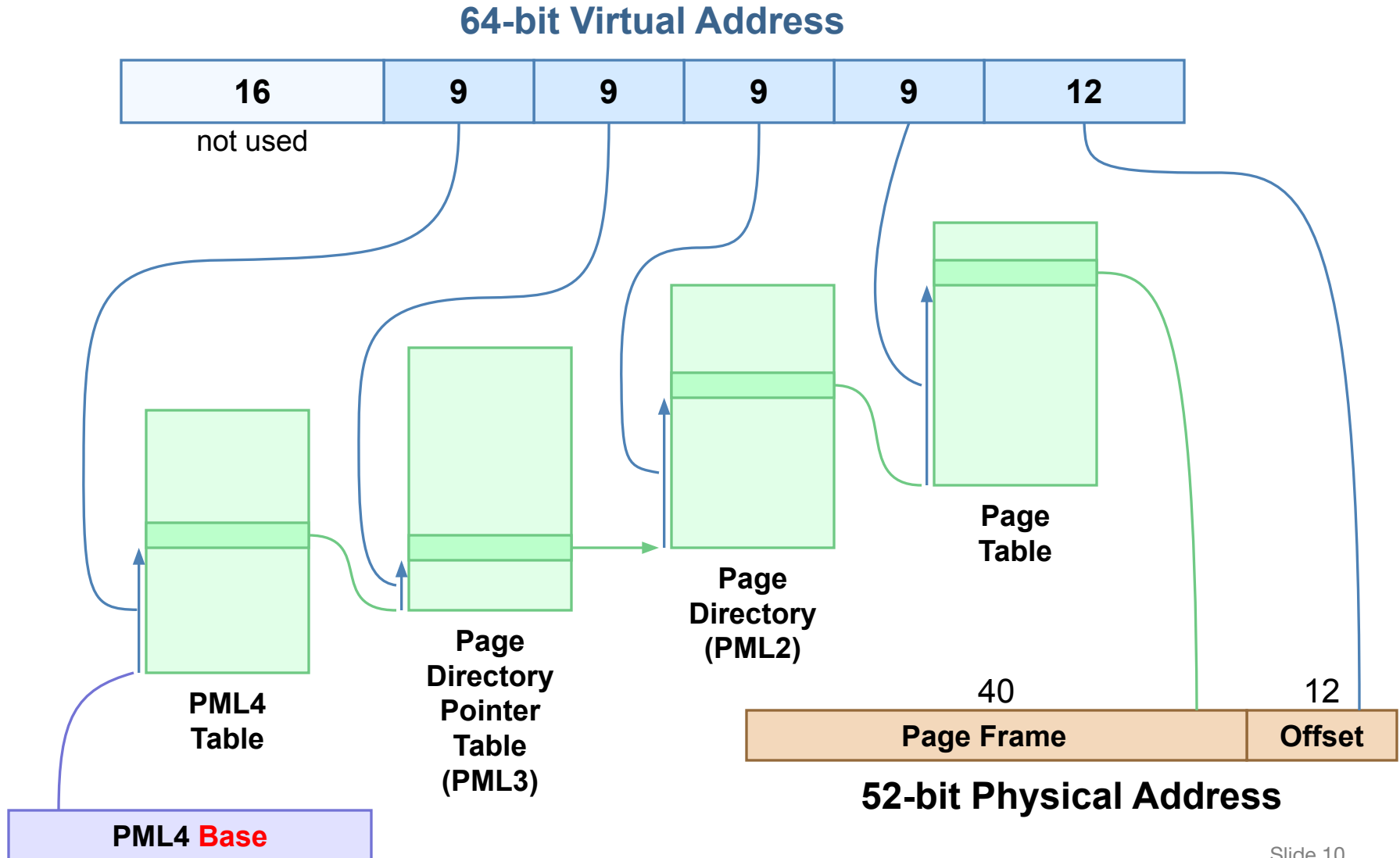
$$2^{64-12} \times 4B = 2^{54} B = 16 PB \quad (peta \Rightarrow tera \Rightarrow giga)!$$

And this is for ONE process!

Multi-level?

- ▶ If 10 *bits* for each level, then $\frac{64-12}{10} = 5$ levels are required.
5 memory access for each address translation!

x86-64 Address Translation



Inverted Page Tables

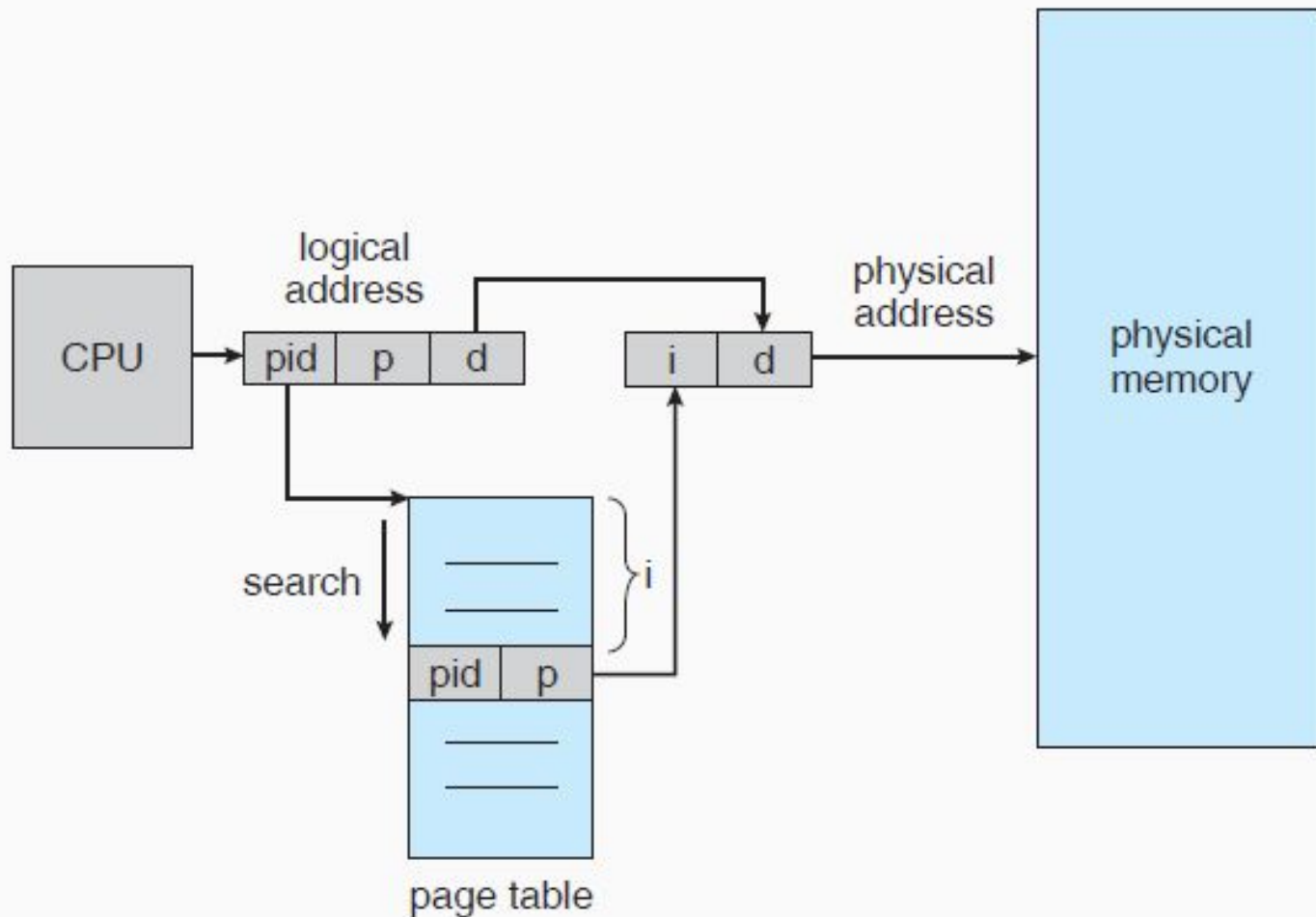
— Index with frame number

Inverted Page Table:

- ▶ One entry for each **physical** frame
 - ▶ The physical frame number is the table index
- ▶ A single **global page table for all processes**
 - ▶ The table is shared — **PID is required**
- ▶ Physical pages are now mapped to virtual — each entry contains a virtual page number instead of a physical one
- ▶ Information bits, e.g. **protection bit**, are as usual

Find index according to entry contents

$(pid, p) \Rightarrow i$



Std. PTE (32-bit sys.): Inverted PTE (64-bit sys.):

page frame address	info
20	12

indexed by page number

pid	virtual page number	info
16	52	12

indexed by frame number

if 2^{20} entries, 4B each
 then $SIZE_{page\ table} = 2^{20} \times 4 = 4MB$
 (for each process)

if assuming

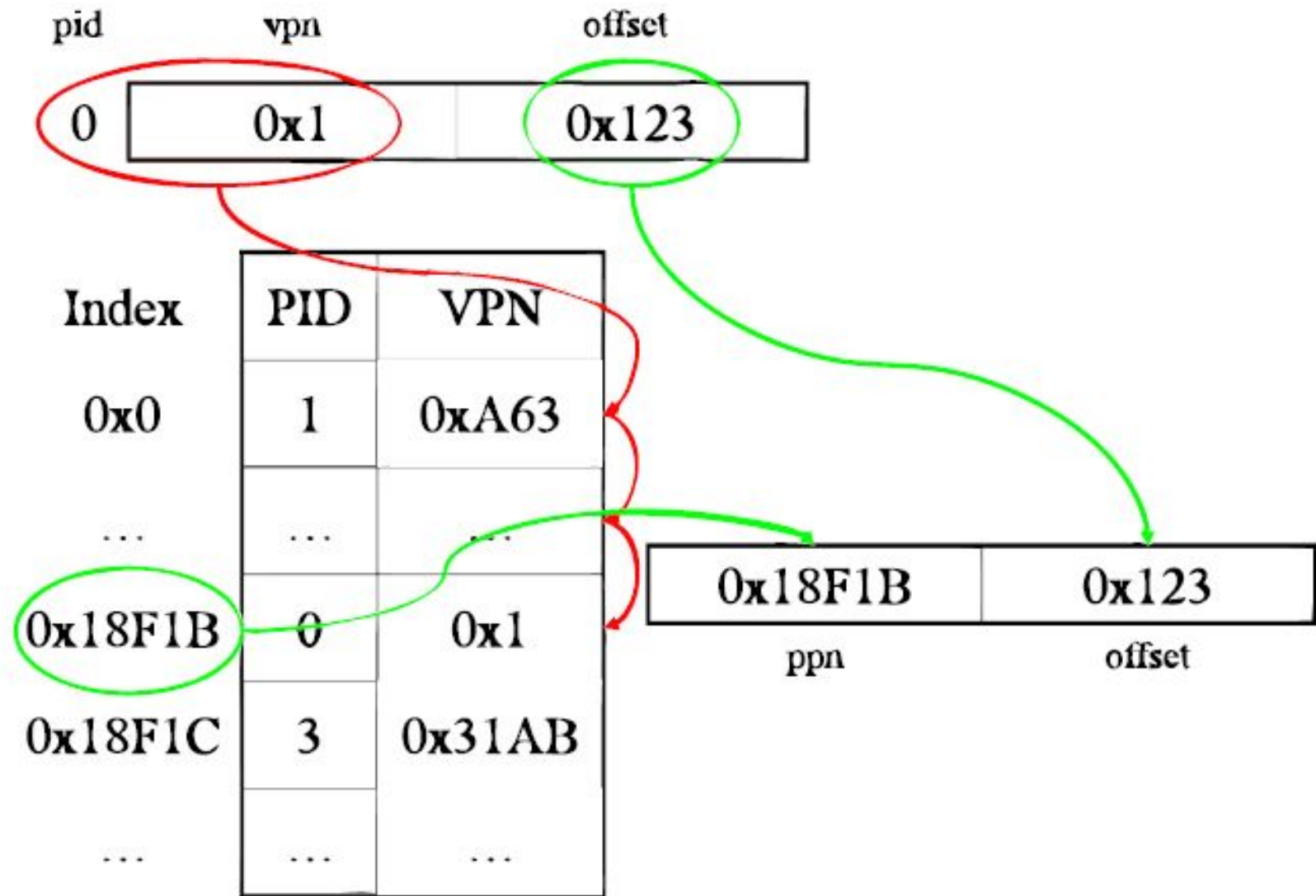
- ▶ 16 bits for PID
- ▶ 52 bits for virtual page number
- ▶ 12 bits of information,

then each entry takes
 $16 + 52 + 12 = 80\ bits = 10\ bytes$

if physical mem = 1G ($2^{30} B$), and
 page size = 4K ($2^{12} B$), we'll
 have $2^{30-12} = 2^{18}$ pages.

then $SIZE_{page\ table} = 2^{18} \times 10 B = 2.5 MB$
 (for all processes)

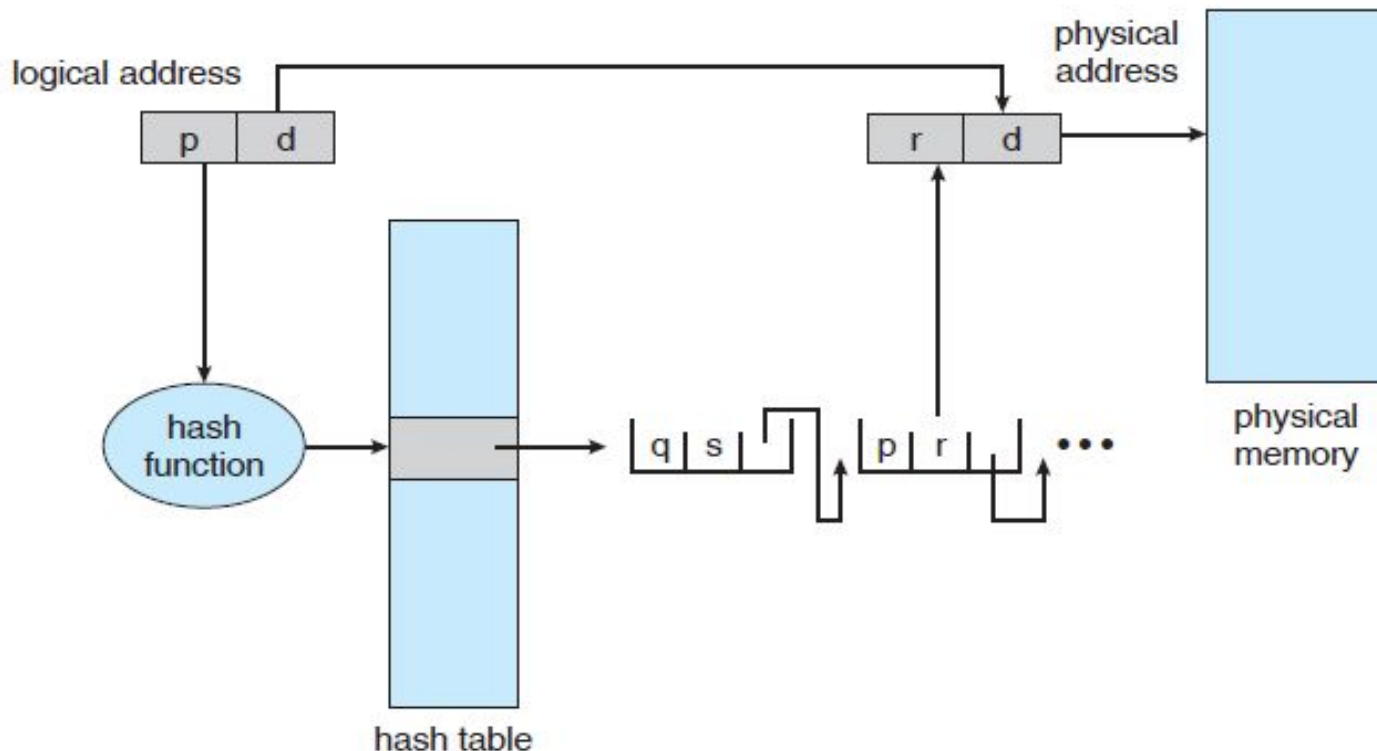
Inefficient: require searching the entire table



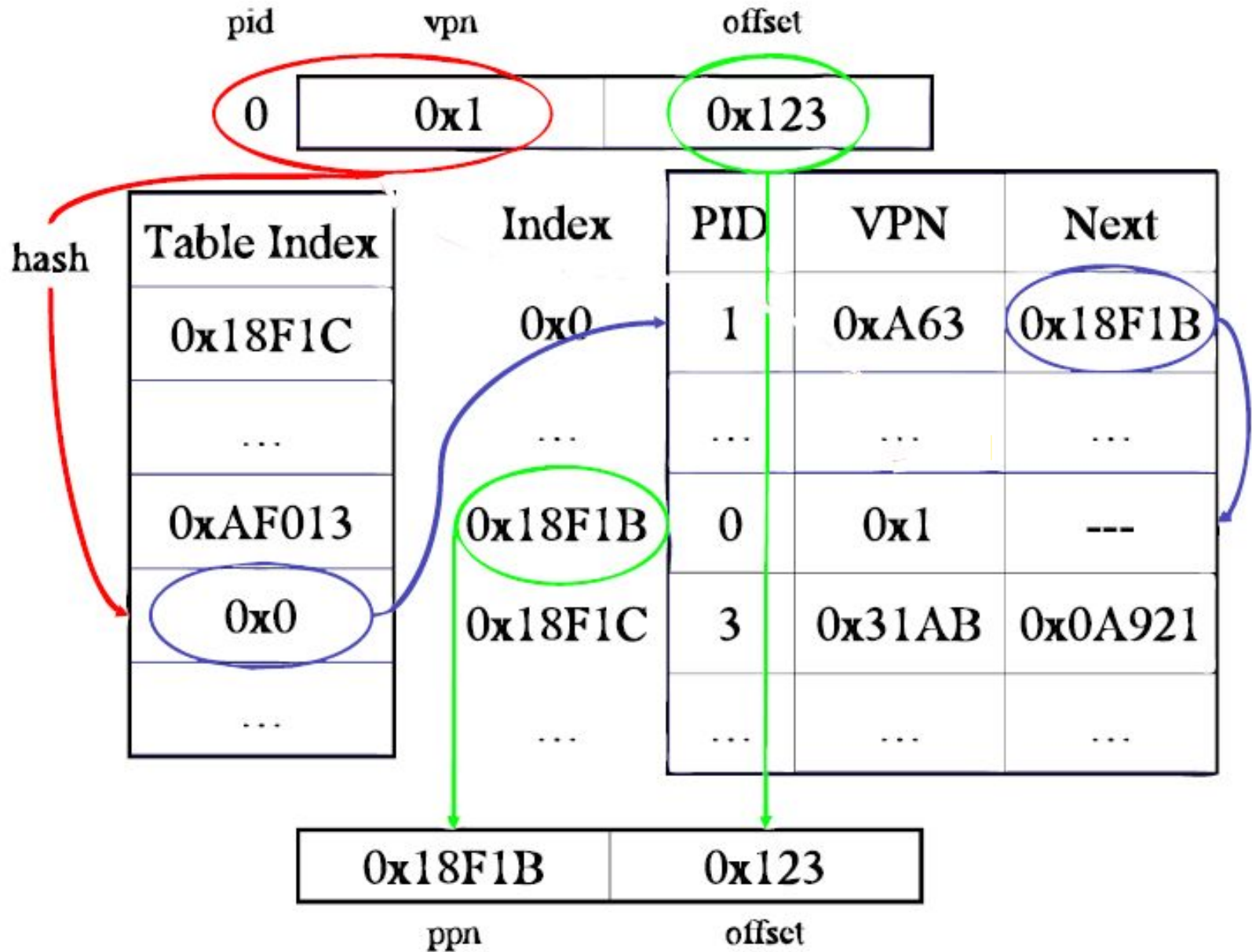
Hashed Inverted Page Tables

A **hash anchor table** — an extra level before the actual page table

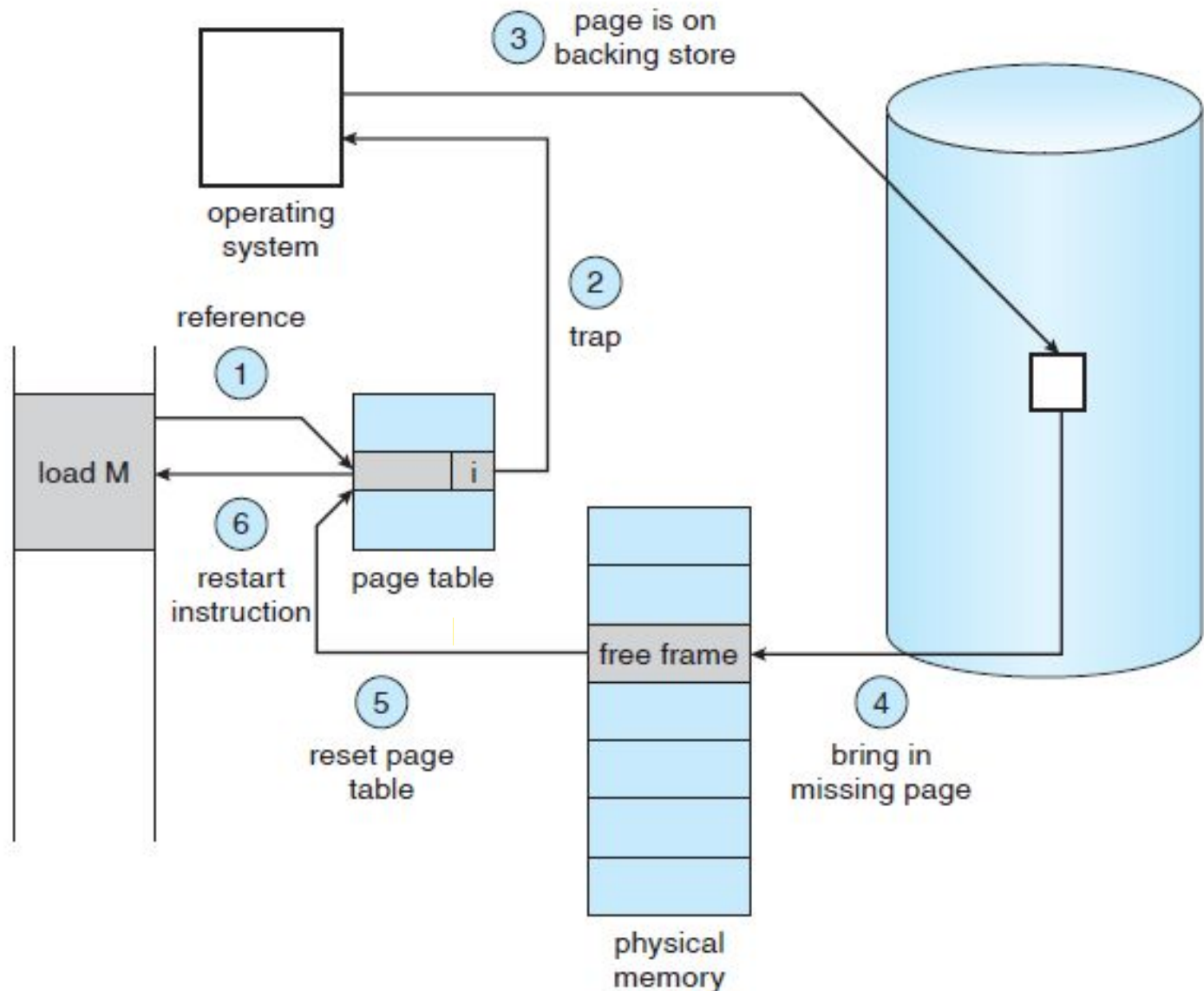
- ▶ maps $\begin{matrix} \text{process IDs} \\ \text{virtual page numbers} \end{matrix} \Rightarrow \text{page table entries}$
- ▶ Since collisions may occur, the **page table must do chaining**



Hashed Inverted Page Table



Restoring State...Plz be Attentive



Page Replacement

- Upon page fault:
 - Find a free page frame.
 - Fetch disk block into page frame.
 - When disk read finishes, add vm mapping for the page (change page table entries).
- Find a free page frame
 - If there is a free page frame, use it
 - Otherwise, select a page to be replaced (page replacement algorithm), and
 - write the selected page to the disk if necessary

Issue: Eviction

- Goal: kick out the page that's **least useful**
 - Pages that will not be used immediately, frequently
- Better not to choose a page that will be used immediately, frequently
 - will probably need to be brought back in soon
- Clean pages (not modified)
 - Hardware has a dirty bit for each page frame
 - Need to write back to disk if a “dirty” page is evicted
- Problem: how do you determine *least useful*?
 - Different heuristics

Optimal Page Replacement Algorithm

- Replace a page whose **next** use will occur **farthest** in the future.
- Should have least number of page faults
- The only problem is
 - Unrealizable
 - Why? 2 minutes 
 - needs a time machine

Optimal Page Replacement Algorithm

- However, can be used to evaluate other strategies

Access	Hit/Miss?	Evict	Resulting Cache State	$\frac{Hits}{Hits+Misses}$
0	Miss		0	$\frac{6}{6+5}$ or 54.6%.
1	Miss		0, 1	
2	Miss		0, 1, 2	
0	Hit		0, 1, 2	
1	Hit		0, 1, 2	
3	Miss	2	0, 1, 3	
0	Hit		0, 1, 3	
3	Hit		0, 1, 3	
1	Hit		0, 1, 3	
2	Miss	3	0, 1, 2	
1	Hit		0, 1, 2	

Not Recently Used Page Replacement Algorithm

- Each page has Reference bit, Modified bit
 - bits are set when page is referenced, modified
- At a certain fixed time interval, the clock interrupt triggers and clears the referenced bit of all the pages
 - only pages referenced within the current clock interval are marked with a referenced bit.

Not Recently Used Page Replacement Algorithm

When a page fault occurs...

Categorize each page...

<u>Class 1:</u>	Referenced = 0	Dirty = 0
<u>Class 2:</u>	Referenced = 0	Dirty = 1
<u>Class 3:</u>	Referenced = 1	Dirty = 0
<u>Class 4:</u>	Referenced = 1	Dirty = 1

Choose a page from class 1.

If none, choose a page from class 2.

If none, choose a page from class 3.

If none, choose a page from class 4.

implies that a modified but **not** referenced (at least within last clock interval) page is less important than a not modified page that is intensely referenced.